

Задача 1. Симуляция полива комнатных растений (Базовый `asyncio.gather`)

Контекст: У вас дома есть 3 цветка: Фикус, Кактус и Орхидея. Полив каждого цветка (наполнение почвы влагой) занимает определенное время.

Задание:

1. Напишите асинхронную функцию `water_flower(name, duration)`, которая принимает имя цветка и время, необходимое на его полив.
2. Функция должна выводить сообщение: "[Полив] Начинаю поливать {name}...", делать паузу на `duration` секунд с помощью асинхронного сна, а затем выводить: "[Полив] {name} успешно полит!" и возвращать строку "{name} готов".
3. В главной функции `main` запустите конкурентно полив всех трех растений одновременно, используя `asyncio.gather`:
 - Фикус (2 секунды)
 - Кактус (1 секунда)
 - Орхидея (3 секунды)
4. Выведите на экран итоговый список результатов, который вернет `asyncio.gather`, а также замерьте общее время выполнения программы. Она должна завершиться примерно за 3 секунды (а не за 6).

Задача 2. Скачивание обновления для игровой приставки (Работа с `asyncio.create_task`)

Контекст: Вы сели играть в приставку, но игра требует загрузить фоновое обновление. Вы не хотите сидеть и ждать окончания загрузки, а хотите запустить скачивание в фоне и пока заняться чем-то еще (например, разложить вещи на столе).

Задание:

1. Напишите корутину `download_update()`, которая имитирует загрузку файла в течение 4 секунд. Каждую секунду она должна выводить в консоль: "[Загрузка] Скачано ...%" (можно сделать

через цикл `for` на 4 итерации с паузой в 1 секунду). В конце она возвращает "Обновление установлено".

2. В функции `main` оберните корутину в задачу через `asyncio.create_task()`, чтобы загрузка сразу стартовала в фоне.
3. Сразу после этого (не используя `await` для задачи загрузки) симитируйте свое действие: сделайте `await asyncio.sleep(2)` и выведите сообщение "[Я] Разложил вещи на столе...".
4. После этого проверьте статус задачи с помощью метода `.done()`. Выведите в консоль, завершилась ли загрузка.
5. В самом конце примените `await` к задаче загрузки, чтобы дождаться её окончания и забрать финальный результат.

Задача 3. Сгоревшая запеканка (Безопасная обработка исключений)

Контекст: Вы решили приготовить на ужин три блюда одновременно через `asyncio.gather`. Но что-то пошло не так: духовка сломалась, и запеканка сгорела, вызвав программную ошибку. Вам нужно сделать так, чтобы из-за одного сгоревшего блюда вы не потеряли информацию об остальных (успешно приготовленных).

Задание:

1. Создайте корутину `cook_dish(name, duration, fail=False)`. Если флаг `fail=True`, корутина через `duration` секунд должна вызывать исключение `RuntimeError("Блюдо сгорело!")`. Иначе — возвращать строку `"{name} готов"`.
2. В `main` запустите три задачи через `asyncio.gather`:
 - Салат (1 секунда, `fail=False`)
 - Запеканка (2 секунды, `fail=True`)
 - Компот (3 секунды, `fail=False`)
3. Настройте `asyncio.gather` так, чтобы ошибка внутри одной корутины не прерывала работу остальных, а возвращалась в виде объекта исключения в общем списке результатов (используйте правильный именованный аргумент функции `gather`).

4. С помощью цикла переберите полученные результаты. Если элемент списка является исключением (`isinstance(item, Exception)`), выведите сообщение об ошибке, иначе — выведите отчет об успешном приготовлении блюда.

Задача 4. Будильник с возможностью отмены (Управление через `.cancel()`)

Контекст: Вы ставите долгий будильник (таймер сна). Но если вы проснулись раньше сами, вы должны иметь возможность подойти и вручную отменить этот будильник, чтобы он не звенел.

Задание:

1. Напишите корутину `alarm_clock()`, которая должна «спать» 10 секунд. Внутри нее предусмотрите обработку исключения `asyncio.CancelledError` через блок `try...except`. Если будильник отменили, должно выводиться: "[Будильник] Отменен пользователем". Если 10 секунд прошли успешно — "[Будильник] Динь-динь! Пора вставать!".
2. В функции `main` запустите будильник как фоновую задачу (`create_task`).
3. Напишите, что вы спите: `await asyncio.sleep(3)`.
4. Представьте, что вы проснулись сами через 3 секунды. Вызовите метод `.cancel()` у вашей задачи будильника.
5. Обязательно добавьте `await` для задачи будильника после её отмены внутри блока `try...except`, чтобы корректно завершить корутину и перехватить финальное исключение `CancelledError` в функции `main`.